Vyro OS: Design and Implementation of a From-Scratch 64-bit Operating System

Gaurav Ganesh Teegulla

June 2026

Abstract

We present Vyro OS, a 64-bit operating system constructed entirely from scratch on bare x86-64 hardware without any existing OS libraries, libc, or runtime support. Vyro OS progresses through eight major development iterations, beginning with a 512-byte BIOS boot sector and culminating in a full graphical desktop running in QEMU with working mouse, keyboard, sound, and networking. The system implements 25 subsystems from first principles: a physical memory manager, a first-fit heap allocator, a cooperative-preemptive scheduler, an ELF64 user-mode loader, a virtual file system with FAT32 backend, a complete TCP/IP stack with TLS 1.3, a double-buffered glassmorphic compositor, nine cryptographic primitives verified against published RFC test vectors, and a PS/2 plus VMware-backdoor mouse driver. Five production-grade bugs diagnosed through runtime instrumentation are documented: a BSS overwrite detected via CR2 capture, an IRQ-state deadlock in the sleep primitive, an i8042 output-buffer race, a framebuffer initialisation race, and a mouse protocol mismatch caused by the QEMU usb-tablet device. Version 7 introduced a tri-path product strategy targeting three delivery vehicles simultaneously. The paper describes all architectural decisions, the full bug history, and an honest assessment of remaining work against a production daily-driver OS.

Keywords: operating systems, microkernel, x86-64, QEMU, memory management, double-buffered compositor, TLS 1.3, SMP, VMware backdoor, BSS corruption, glassmorphism, i8042, FAT32, ELF64, ring-3 user mode

Contents

1. Introduction
2. System Architecture
   2.1 Address Space Layout
   2.2 Kernel Subsystem Map
3. Boot Process
   3.1 Long Mode Enable Sequence

- 3.2 VBE Framebuffer Setup
- 3.3 Boot Sequence Timeline

4. Memory Management
   - 4.1 Physical Memory Manager
   - 4.2 Kernel Heap
   - 4.3 Page Table Management
5. Interrupts and Scheduling
   - 5.1 Interrupt Descriptor Table
   - 5.2 IRQ-aware sleep_ms
   - 5.3 Cooperative Scheduler
   - 5.4 Context Switch
   - 5.5 PIT Timer Accuracy
6. Shell and Built-in Commands
7. Virtual File System and FAT32
   - 7.1 FAT32 Backend
   - 7.2 Kernel Log
8. Hardware Driver Subsystem
   - 8.1 AHCI and NVMe
   - 8.2 RTL8139 and E1000
9. Graphics Compositor
   - 9.1 Drawing Primitives
   - 9.2 Glassmorphism Effect
   - 9.3 VMware Backdoor Mouse
   - 9.4 BSS Canary Instrumentation
10. Networking Stack
    - 10.1 TCP State Machine
    - 10.2 TLS 1.3 Key Schedule
11. Security Subsystem
    - 11.1 Cryptographic Primitives
    - 11.2 Trust Anchor Store
12. User Mode and ELF64 Loader
    - 12.1 System Calls
13. IPC Subsystem
14. Development Methodology
    - 14.1 Git Discipline
    - 14.2 Build System
    - 14.3 Lines of Code
15. Performance and System Characteristics
16. Tri-Path Product Architecture
    - 16.1 Path A — Ubuntu Remix
    - 16.2 Path B — Linux Core
    - 16.3 Path C — Microkernel
17. Production Bug Analysis
    - 17.1 BSS Corruption

17.2 sleep_ms Deadlock
17.3 First Keystroke Eaten
17.4 GUI Silent Non-Launch
17.5 Mouse Clicks Non-Functional
18. Evaluation
19. Related Work
20. Future Work
21. Conclusion
Acknowledgements
References

1. Introduction

Building an operating system from scratch is one of the most demanding exercises in systems programming. Unlike application development, OS construction offers no safety net: a misaligned page-table entry, a wrong interrupt vector offset, or an unmasked IRQ all produce a silent triple-fault rather than a helpful stack trace. Vyro OS was started with one guiding constraint: use no existing runtime, library, or OS support whatsoever. Every subsystem — the bootloader, GDT, IDT, physical memory manager, heap, scheduler, VFS, TCP/IP stack, TLS 1.3 implementation, desktop compositor, and all nine cryptographic primitives — was written from first principles in x86-64 assembly and C.

The project is calibrated against the cfenollosa OS tutorial specification (17 lessons covering a basic boot sector through a minimal shell) but quickly grew far beyond that scope. By version 8 the codebase contains approximately 53,000 lines across 201 source files and implements subsystems that are typically the subject of dedicated research papers: a constant-time ECDSA P-256 verifier, a ChaCha20-Poly1305 AEAD primitive, and a TLS 1.3 key schedule verified against RFC 8448 test vectors. A 155-tag git history documents every working state of the system from the first boot sector to the current graphical desktop.

The paper is organised as follows. Section 2 describes the overall system architecture and memory layout. Section 3 covers the boot process. Section 4 presents memory management. Section 5 documents interrupt handling and scheduling. Section 6 describes the shell. Section 7 covers the file system. Section 8 describes hardware drivers. Section 9 presents the graphics compositor. Section 10 covers networking. Section 11 details security and cryptography. Section 12 covers user mode. Section 13 describes IPC. Section 14 presents development methodology. Section 15 documents performance. Section 16 describes the tri-path product architecture. Section 17 documents five production bugs. Section 18 evaluates completeness. Sections 19-21 cover related work, future work, and conclusions.

2. System Architecture

Vyro OS follows a monolithic-with-modules architecture for the Path C microkernel. All kernel subsystems run in ring 0 with a shared address space, while user-mode code runs in ring 3 using hardware-enforced privilege separation via the x86-64 TSS and CPL bits. The kernel occupies the lower 4 GB using identity-mapped page tables established by the bootloader through a 4-level hierarchy (PML4, PDPT, PD, PT).

2.1 Address Space Layout

The physical memory layout is fixed at boot time. The kernel image is loaded at physical 0x10000, the kernel stack sits at 0x90000, the BIOS 8x16 font is copied to 0x80000, and the LFB address is stored at 0x0500 for later retrieval. The heap spans 0x500000 to 0xD00000 (8 MB). The compositor back-buffer is pinned at 0x1000000 (16 MB) to prevent BSS corruption from reaching the pixel buffer.

Region	Physical Address	Size	Purpose
LFB address store	0x0500	4 B	Bootloader writes VBE PhysBasePtr here
Bootloader stage 2	0x0600 – 0x7FFF	30 KB	Disk read, VBE mode-set, GDT, mode switch
BIOS font	0x80000	4 KB	8x16 glyph data for text and compositor
Kernel stack	0x90000	64 KB	Initial ring-0 RSP
Kernel image	0x10000 – 0x4FFFF	~256 KB	Compiled kernel binary
Heap	0x500000 – 0xCFFFFF	8 MB	kmalloc / kfree arena
Compositor buffer	0x1000000	2.36 MB	1024x768x3 back-buffer (fixed address)
Free RAM	0x1300000+	~46 MB	Physical memory manager bitmap pool

Table 1. Physical memory layout at boot time.

2.2 Kernel Subsystem Map

The kernel is decomposed into 12 major subsystems: boot (boot.asm and kernel_entry.asm), hardware abstraction (GDT, IDT, PIC, LAPIC), memory (PMM, heap, page tables), storage (ATA, AHCI, NVMe, FAT32, VFS, block layer, partition table), scheduling (cooperative round-robin with 20 ms preemption), networking (ARP, IPv4, IPv6, UDP, TCP, DHCP, DNS, TLS 1.3), graphics (framebuffer, compositor v2, theme, widgets, GUI), security (SHA-256, ChaCha20, Poly1305, HMAC-HKDF, X25519, ECDSA, RSA, X.509, TLS, CSPRNG, trust anchors), user-mode (ELF64, syscall, ring-3 trampoline), IPC (pipes, message queues, signals), applications (12 built-in apps), and system utilities (shell, klog, power, package manager).

3. Boot Process

The boot process is entirely BIOS-based. The first 512-byte sector performs stack setup at 0x90000, A20 gate enable via port 0x92, INT 0x13 extended read (function 0x42) to load kernel sectors, INT 0x10 VBE mode-set to 1024x768x24bpp with the returned PhysBasePtr stored at 0x0500, GDT load via lgdt, and CR0.PE enable to enter protected mode.

3.1 Long Mode Enable Sequence

A 32-bit stub enables long mode by setting CR4.PAE, loading a temporary PML4 that identity-maps 0 to 4 GB, writing EFER.LME via rdmsr/wrmsr, enabling paging with CR0.PG, and jumping to the 64-bit kernel entry point in kernel_entry.asm.

```
; 32-bit stub
mov  eax, cr4
or   eax, 0x20         ; CR4.PAE
mov  cr4, eax
mov  eax, pml4_table   ; identity-map 0-4 GB
mov  cr3, eax
mov  ecx, 0xC0000080   ; EFER MSR
rdmsr
or   eax, 0x100        ; EFER.LME
wrmsr
```

```
mov  eax, cr0
or   eax, 0x80000001  ; CR0.PG | CR0.PE
mov  cr0, eax
jmp  0x08:long_mode_entry
```

3.2 VBE Framebuffer Setup

Setting a linear framebuffer from a 16-bit BIOS environment requires the AX=0x4F02 VBE call with bit 14 (LFB flag) set in BX. The VBE mode information block at physical 0x7E00 contains the 32-bit PhysBasePtr, which the bootloader stores at 0x0500 before entering protected mode. The kernel reads this address via inline assembly using *movl (0x500), %0* with a register output constraint to prevent the compiler from treating a zero-page read as undefined behaviour.

3.3 Boot Sequence Timeline

Stage	Subsystem	Key action
1	GDT + TSS	Ring 0/3 segments; TSS with ring-3 RSP for syscall
2	IDT	256 gate descriptors; ISR stubs; exception + IRQ vectors
3	PIC (8259A)	Remap master to 0x20-0x27, slave to 0x28-0x2F
4	Keyboard	i8042 OBF drain, IRQ1 register, unmask
5	PIT Timer	100 Hz; divisor 11931; IRQ0; IRQ-aware sleep_ms
6	PMM	Bitmap over E820 free regions; 62 MB on QEMU
7	Heap	First-fit coalescing; 8 MB arena; magic cookie check
8	VFS + FAT32	In-memory vnode tree; FAT32 read-only ATA backend
9	Scheduler	Cooperative RR; 20 ms quantum; context switch in asm
10	Syscall gate	INT 0x80 DPL-3 gate; 12 syscalls; ring-3 isolation
11	Networking	ARP, IPv4/v6, UDP, TCP, DHCP, DNS, TLS 1.3
12	Crypto tests	AEAD, X.509, HKDF, X25519, TLS RFC-vector verify
13	Boot chime	tunes_play_boot() via PC speaker; safe after vC.6.15
14	GUI	fb_probe(), compositor init, VMware mouse, render loop

Table 2. Boot sequence — 14 stages from GDT to GUI render loop.

4. Memory Management

4.1 Physical Memory Manager

The PMM uses a flat bit-array where each bit represents one 4 KB page frame. At boot time pmm_init() marks all frames as used, then walks the BIOS E820 memory map to mark usable regions as free, minus the kernel image and all reserved I/O regions below 1 MB. pmm_alloc() scans for the first zero bit and sets it; pmm_free() clears the bit. A first-free-hint is maintained to reduce average alloc scan from O(N) to O(1) under sequential allocation patterns, which dominates during kernel initialisation.

4.2 Kernel Heap

The 8 MB heap uses explicit free-list allocation. Each block is preceded by a 16-byte header containing size, an in-use flag, and a magic cookie (0xDEADBEEF) for corruption detection. kfree() validates the magic cookie before marking a block free, then attempts forward and backward coalescing. The heap was sized at 8 MB after profiling showed a peak allocation of approximately 4.2 MB under worst-case TLS plus 12-app load.

4.3 Page Table Management

The bootloader's temporary PML4 identity-maps the entire 0-4 GB range using 2 MB huge pages (PDE.PS=1). A future version will introduce per-process page tables with a kernel mapping in the upper half and per-process user mappings in the lower half, enabling KASLR and W^X enforcement. The groundwork (pmm_alloc, page table structures in pmm.h) is already in place.

5. Interrupts and Scheduling

5.1 Interrupt Descriptor Table

The IDT is a 256-entry array of 16-byte gate descriptors. Vectors 0-31 are CPU exceptions; vectors 32-47 are hardware IRQs remapped from the 8259A PIC; vector 0x80 is the system call gate with DPL=3 (callable from ring 3). All 256 ISR entry stubs are generated by a macro in isr_stubs.asm that pushes the interrupt number plus a dummy error code and jumps to the common C dispatch handler in isr.c.

5.2 IRQ-aware sleep_ms

A persistent bug across versions 5-7 caused sleep_ms() to deadlock before the sti instruction in kernel_main(). The original implementation called hlt inside the spin loop unconditionally; hlt halts the CPU until the next interrupt, but if RFLAGS.IF = 0 no interrupt will ever arrive. Version vC.6.15 introduced irq_enabled() which reads RFLAGS bit 9 via *pushfq; popq %0* with a memory constraint, and sleep_ms() now substitutes the pause instruction when IF = 0. Two earlier fix attempts triple-faulted because the missing memory clobber caused the compiler to emit an incorrect frame pointer offset for subsequent stack accesses.

5.3 Cooperative Scheduler

The scheduler implements cooperative round-robin multitasking. Each task holds a task_t struct: uint64_t rsp, uint64_t* stack (heap-allocated 16 KB), uint8_t state

(RUNNING/READY/BLOCKED/ZOMBIE), uint32_t quantum_ticks, and char name[32]. sched_tick() increments the running task's quantum counter; at 2 ticks (20 ms) sched_yield() is invoked from IRQ context.

5.4 Context Switch

The context switch in switch.asm saves and restores the seven callee-saved registers (RBX, RBP, R12-R15) plus RSP. Because GCC's x86-64 ABI already saves caller-saved registers, the switch touches only 7 registers per task, making it extremely lightweight. The old task's RSP is stored at [RDI] and the new task's RSP is loaded from [RSI].

```
sched_switch:
  push rbx   / push rbp   / push r12
  push r13   / push r14   / push r15
  mov  [rdi], rsp          ; save old RSP
  mov  rsp,   [rsi]        ; load new RSP
  pop  r15   / pop  r14   / pop  r13
  pop  r12   / pop  rbp   / pop  rbx
  ret
```

5.5 PIT Timer Accuracy

The PIT channel 0 divisor of 11931 gives a tick period of 9.999 ms (100 Hz). timer_uptime_ms() uses integer arithmetic (tick_count * 1000) / timer_freq to avoid floating-point in kernel context. All sleep_ms() calls are accurate to within one tick period (10 ms) because the loop exits at the first tick boundary after the target time.

6. Shell and Built-in Commands

The Vyro OS shell (shell.c, approximately 2,800 lines) provides command history (UP/DOWN arrows, 64-entry circular buffer), inline editing, and approximately 40 built-in commands dispatched via a sorted command table with binary search.

Category	Commands
File system	ls, cat, mkdir, rm, touch, cp, mv, write, hexdump
System info	dmesg, mem, uptime, cpuinfo, pci, lspci, uname
Networking	ping, dhcp, dns, curl, http, ifconfig, netstat
Security	sha256, cert, tls, whoami, useradd, passwd
Graphics	gui, theme, wallpaper, screenshot
Audio	beep, scale, arpeggio, alert
Package mgr	vyropkg install, list, remove, update
User mode	exec, ps, kill, yield, fork
Power	shutdown, reboot, sleep, lock
Debug	peek, poke, prof, canary, kmalloc_test

Table 3. Shell built-in commands grouped by category.

7. Virtual File System and FAT32

The VFS provides a POSIX-inspired interface (vfs_open, vfs_read, vfs_write, vfs_close, vfs_mkdir, vfs_readdir, vfs_stat, vfs_unlink) over a tree of vnode structs. Each vnode contains a name, type (file or directory), data pointer, size, and a vtable of function pointers for backend operations.

7.1 FAT32 Backend

The FAT32 driver reads the Volume Boot Record to locate the FAT region start, FAT count, sectors per FAT, root cluster, and sectors per cluster. Directory enumeration walks 32-byte directory entries in the root cluster chain. A critical macro-indirection bug (vC.6.10.5) caused the FAT32_EOC_MIN termination check to be mis-parsed due to operator precedence. The expression *0x0FFFFFF8 <= val* required explicit parentheses around the masked val argument to avoid an incorrect parse by the C preprocessor.

7.2 Kernel Log

klog.c implements a circular 256-entry string log stored in BSS. Every ok() call during boot logs its message; the dmesg shell command dumps the ring buffer. This proved invaluable during bug hunting: after a hang, running dmesg on the next boot showed exactly which subsystem was the last to initialise, narrowing the search space from approximately 40 subsystems to 1-2.

8. Hardware Driver Subsystem

Vyro OS implements detection and partial initialisation drivers for twelve hardware controllers. Drivers are structured in three tiers: *detection-only* (probe PCI bus, read capability registers), *partial* (detection plus internal controller state setup), and *full* (detection, setup, and working I/O).

Controller	Tier	Key details
AHCI SATA (ahci.c)	Partial	PCI class 01/06/01; ABAR map; GHC.AE; 32-port enum via PORT_SSTS
NVMe (nvme.c)	Partial	PCI class 01/08/02; 64-bit BAR; CAP.MQES+DSTRD; Admin Queue
Intel E1000 (e1000.c)	Partial	12 PCI device IDs; BAR0 MMIO; MAC from RAL/RAH
RTL8139 (rtl8139.c)	Full	TX/RX rings; interrupt-driven; real packet I/O in QEMU
xHCI USB 3.0 (xhci.c)	Partial	Capability regs; halt/reset; DCBAA; Command + Event Ring
ATA PIO (ata.c)	Full	28-bit LBA; PIO mode 0 via 0x1F0 ports; read/write
PS/2 Keyboard	Full	IRQ1; scan code set 1; i8042 OBF drain; shift/caps/extended
PS/2 Mouse + VMware	Full	IRQ12 + VMware backdoor; absolute + relative modes
PIT 8253 (timer.c)	Full	Channel 0 mode 3 (100 Hz); channel 2 for PC speaker
PC Speaker	Full	PIT channel 2 + port 0x61; on/off; frequency control
RTC MC146818 (rtc.c)	Full	CMOS ports 0x70/0x71; year/month/day/hour/min/sec

Controller	Tier	Key details
Local APIC (lapic.c)	Partial	MSR IA32_APIC_BASE; MMIO map; SVR enable; TPR clear

Table 4. Hardware driver inventory — 12 drivers across three implementation tiers.

8.1 AHCI and NVMe

The AHCI driver maps the HBA Memory Registers via PCI BAR5 (ABAR), sets GHC.AE to take ownership from legacy IDE mode, then scans PORT_SSTS for each of the 32 possible ports checking Device Detection (DET=3, device present and PHY active). The NVMe driver reads the Controller Capabilities (CAP) register to extract MQES and DSTRD, allocates submission and completion queues, and waits for CSTS.RDY after writing CC.EN.

8.2 RTL8139 and E1000

The RTL8139 driver is the only NIC driver at full tier, providing real transmit and receive in QEMU. The E1000 driver detects 12 Intel Gigabit device IDs and reads the MAC address from RAL/RAH registers populated by the EEPROM, but defers full DMA ring setup to a future version.

9. Graphics Compositor

The compositor (compositor.c, over 600 lines) implements double-buffered rendering. All draw operations target a DRAM back-buffer; comp_present() blits the complete 1024x768x3-byte buffer to the linear framebuffer in a single loop. The back-buffer occupies 2,359,296 bytes pinned at physical 0x1000000 and is accessed via a static const pointer in .rodata.

9.1 Drawing Primitives

The compositor provides: comp_pixel (single pixel), comp_rect (filled rectangle), comp_border (1-pixel outline), comp_shadow (4-pixel soft drop shadow), comp_glyph and comp_text (BIOS 8x16 font at 0x80000), comp_gradient_v (vertical linear gradient), comp_blur_rect (box blur with configurable passes), comp_tint_rect (alpha blend with accent colour), comp_rounded_rect (corner-clipped rectangle), and comp_glass_panel (combined blur, tint, and border for glassmorphism).

9.2 Glassmorphism Effect

comp_glass_panel() implements a four-step frosted-glass effect without GPU hardware. Step 1 reads the existing back-buffer pixels in the target region. Step 2 applies a box blur with the specified number of passes, replacing each pixel with the arithmetic mean of its 3x3 neighbourhood. Step 3 blends the blurred result with an accent colour tint at the specified opacity using *out = (blurred * (255 - alpha) + tint * alpha) / 255* per channel. Step 4 draws a 1-pixel border.

9.3 VMware Backdoor Mouse

QEMU's usb-tablet device delivers absolute screen coordinates via the VMware backdoor protocol at I/O port 0x5658 rather than PS/2 relative packets. The protocol uses EAX=0x564D5868 (magic

VMXh), ECX=command, EDX=0x5658 and a single *inl %%dx, %%eax* instruction. vmmouse_enable() sends ABSPTR_COMMAND (0x27) with the ENABLE magic; vmmouse_poll() reads four ABSPTR_DATA (0x29) words: X, Y, Z-axis, and buttons. X and Y are scaled to screen pixels by *(raw * screen_dim) >> 16*.

9.4 BSS Canary Instrumentation

Version vC.6.14 added two 64-bit sentinel values (0xDEADC0DEDEADC0DE) in BSS flanking the compositor globals. comp_canary_ok() checks both sentinels each render frame. comp_canary_dump() prints which sentinel was stomped to the VGA text console on the frame corruption is first detected, identifying whether the OOB write underflows or overflows the compositor globals.

10. Networking Stack

The networking stack is a full in-kernel implementation spanning ten protocol layers. The RTL8139 driver provides real transmit and receive on QEMU's emulated NIC.

Layer	File	Features
Ethernet	rtl8139.c	TX/RX rings, MAC filter, promiscuous mode
ARP	arp.c	Request/reply, ARP cache, gratuitous ARP
IPv4	net.c	Fragmentation, ICMP echo reply, routing table
IPv6	ipv6.c	Link-local (EUI-64), ICMPv6 echo reply, NDP stub
UDP	udp.c	Port dispatch table, checksum, sendto/recvfrom
TCP	tcp.c	Full RFC 793 state machine, Nagle, checksum
DHCP	dhcp_real.c	DISCOVER/OFFER/REQUEST/ACK, option parsing
DNS	dns_real.c	A and AAAA queries, response parser, cache
HTTP	http.c	GET/HEAD, chunked response, VyroBrowser renderer
TLS 1.3	tls.c	Key schedule RFC 8448 verified; AEAD encrypt/decrypt

Table 5. Networking stack — ten protocol layers implemented from scratch.

10.1 TCP State Machine

The TCP implementation follows RFC 793 with a complete 11-state machine: CLOSED, LISTEN, SYN-SENT, SYN-RECEIVED, ESTABLISHED, FIN-WAIT-1, FIN-WAIT-2, CLOSE-WAIT, CLOSING, LAST-ACK, TIME-WAIT. Sequence numbers, acknowledgement numbers, window scaling, and Internet checksums are correctly computed. Congestion control and selective acknowledgement are deferred to a future version.

10.2 TLS 1.3 Key Schedule

The TLS 1.3 implementation verifies its key schedule against the RFC 8448 Section 3 test vectors. HKDF-Extract and HKDF-Expand-Label are implemented on top of HMAC-SHA256 (itself verified against RFC 4231). The Derive-Secret function uses HKDF-Expand-Label with a transcript hash. A TLS RX buffer stall bug (fixed in v4) caused the parser to re-process the Finished message on every read because the cursor was not advanced past finished_len bytes after successful MAC verification.

11. Security Subsystem

All nine cryptographic primitives are implemented from scratch without linking any external library. This was both a design constraint and an educational goal.

11.1 Cryptographic Primitives

Primitive	File	Standard	Self-test
SHA-256	sha256.c	FIPS 180-4	NIST test vector
ChaCha20-Poly1305	aead.c + poly1305.c	RFC 8439	RFC 8439 vectors
HMAC-SHA256 + HKDF	hkdf.c	RFC 5869	RFC 5869 vectors
X25519 ECDH	x25519.c	RFC 7748	RFC 7748 vectors
ECDSA P-256 verify	ecdsa.c	RFC 6979	Deterministic nonce
RSA + PKCS1-v1.5	rsa.c + bignum.c	RFC 8017	4096-bit modexp
RSA-PSS	rsa_pss.c	RFC 8017 §9.1	PSS round-trip
X.509 DER parser	x509.c	RFC 5280	406-byte DER cert
TLS 1.3 key sched.	tls.c	RFC 8448 §3	All derived secrets
CSPRNG	csprng.c	—	RDRAND + RDTSC seed

Table 6. Cryptographic primitives — all verified against published test vectors.

11.2 Trust Anchor Store

trust.c holds a fixed-size array of DER-encoded X.509 certificates loaded at boot: the Vyro self-signed root CA (RSA-2048), a mock ISRG Root X1, DigiCert Global Root, and GlobalSign Root CA. trust_verify() walks the certificate chain from leaf to a trust anchor, checking the RSA-PSS or ECDSA signature at each step and validating the Basic Constraints extension.

12. User Mode and ELF64 Loader

Vyro OS supports ring-3 user-mode processes using the x86-64 hardware privilege separation mechanism. The TSS is loaded with a kernel RSP in RSP0 so that interrupt delivery switches to the kernel stack correctly. The usermode trampoline (usermode.asm) performs an iretq with CS=0x23 (ring-3 code) and SS=0x2B (ring-3 data) to enter user mode.

The ELF64 loader (elf.c) validates the ELF magic, e_machine (0x3E = x86-64), and e_type (ET_EXEC), then iterates PT_LOAD segments, allocates physical pages via pmm_alloc, maps them, copies segment data, and jumps to e_entry via the trampoline. Two user-mode programs (user/init.c and user/hello.c) are compiled to ELF64 and embedded as byte arrays in the kernel image at build time.

12.1 System Calls

Twelve system calls are implemented via INT 0x80 with a DPL=3 gate. The calling convention passes the syscall number in RAX and up to three arguments in RBX, RCX, RDX. Calls implemented: sys_write, sys_exit, sys_getpid, sys_sleep, sys_fork (stub), sys_exec, sys_open, sys_read, sys_close, sys_malloc, sys_free, sys_yield.

13. IPC Subsystem

ipc.c implements three IPC mechanisms: pipes (unidirectional byte streams with a 4 KB kernel buffer), message queues (fixed 256-entry ring buffers of 128-byte messages), and signals (a 64-bit signal mask per task with a default handler table). The GUI uses IPC message queues to dispatch user events from the interrupt handlers to the application render functions, decoupling interrupt context from application context.

14. Development Methodology

Vyro OS is developed under a strict set of self-imposed engineering rules codified in rulz/ and audited in docs/RULZ_COMPLIANCE.md. The most important rules are: (1) go in order — every lesson builds on the previous; (2) test in QEMU before any real hardware attempt; (3) cross-compile always using x86_64-elf-gcc; (4) no libc; (5) interrupts before anything advanced; (6) one feature at a time — build, boot, verify, then continue; (7) read Linux source when stuck on advanced topics.

14.1 Git Discipline

Every completed phase results in an annotated git tag pushed to the public repository. Tags follow a structured naming scheme: vMAJOR.MINOR for meta releases, vC.X.Y for Path C microkernel tags, vA.X.Y for Path A, and vB.X.Y for Path B. Phase numbering is strictly monotonic. Hotfixes receive a third dot (e.g., vC.6.10.3). Commit messages require a single long subject line explaining what changed and why, with no markdown bullets, making git log --oneline readable as a chronological changelog.

14.2 Build System

The Makefile compiles all sources with *x86_64-elf-gcc -O2 -ffreestanding -nostdlib -Wall -Wextra*, links with a custom link.ld placing .text at 0x10000, and produces vyro.bin as a flat binary. The build rule *make build 2>&1 | grep error:* must produce zero output before any tag is created. QEMU is launched with: *-cpu max* (required for RDRAND), *-m 64M*, *-usb -device usb-tablet* (absolute mouse), and PC speaker audio.

14.3 Lines of Code

Directory	Files	Lines	Description
boot/	3	~600	Boot sector, UEFI stub, kernel entry
drivers/	20	~3,200	Keyboard, mouse, timer, PIC, framebuffer, NIC, RTC, speaker
kernel/ (core)	30	~8,000	IDT, GDT, PMM, heap, VFS, scheduler, ELF, syscall, IPC, shell
kernel/ (network)	20	~9,000	ARP, IPv4/6, UDP, TCP, DHCP, DNS, HTTP, TLS, sockets
kernel/ (crypto)	18	~7,500	SHA256, ChaCha20, Poly1305, AEAD, HKDF, X25519, ECDSA, RSA, X.509
kernel/ (gui)	15	~10,500	Compositor, GUI, theme, widgets, wallpaper, icons, 12 apps

Directory	Files	Lines	Description
kernel/ (hardware)	12	~5,000	PCI, AHCI, NVMe, ATA, USB, xHCI, LAPIC, ACPI, SMP, block
user/	3	~200	init.c, hello.c, libvyro.h
path-a/ + path-b/	~30	~3,000	Path A live-build config, Path B Buildroot and 5 apps
docs/ + versions/	~20	~6,000	Architecture, roadmap, compliance audit, version histories
Total	~171</b≋b>~53,000		Across all paths and documentation

Table 7. Lines-of-code breakdown by directory at version 8.0.

15. Performance and System Characteristics

The following measurements were taken under QEMU 8.x on a 2021 Apple MacBook Pro (Apple M1 Pro, 16 GB RAM). All values are approximate.

Metric	Observed	Notes
Boot to shell prompt	~180 ms	QEMU cold start; no disk delay
Boot to GUI desktop	~220 ms	Including compositor init and first render
Boot chime duration	~650 ms	5-note arpeggio via PC speaker
Compositor frame time	~12-15 ms	Full 1024x768 back-buffer blit
kmalloc (8-byte block)	<1 us	First-fit on empty heap; near-O(1)
kmalloc (4 KB block)	<2 us	Occasional coalesce adds ~1 us
SHA-256 (1 KB input)	~18 us	Pure C, -O2, no SIMD
ChaCha20-Poly1305 (1 KB)	~22 us	AEAD encrypt + authenticate
X25519 scalar mult	~1.2 ms	Constant-time Montgomery ladder
TCP handshake latency	~4 ms	Client to QEMU user-mode network
DHCP lease acquisition	~8 ms	DISCOVER to ACK round-trip
ATA PIO sector read	~400 us	28-bit LBA, PIO mode 0

Table 8. Observed performance characteristics — QEMU on Apple M1 Pro.

The compositor frame time of 12-15 ms is dominated by the back-buffer blit (1024x768x3 = 2.36 MB). Adding SSE2 or AVX2 128/256-bit stores would reduce the blit to approximately 3 ms, enabling a true 60 fps render loop. This optimisation requires FXSAVE/FXRSTOR in the interrupt handler path, which is a prerequisite for safe SIMD use in kernel context.

16. Tri-Path Product Architecture

Version 7 introduced a strategic pivot from a single linear development path to three parallel product paths after an honest audit (v6) calculated approximately 116 person-months to a daily-driver OS on a single path.

16.1 Path A — Ubuntu 24.04 Remix

Path A uses Ubuntu 24.04 LTS as a base distribution and replaces the desktop shell, branding, theme, boot splash, default applications, and APT repository with Vyro-specific versions. The build system uses Canonical's live-build tool and a GitHub Actions CI pipeline that produces amd64 and arm64 ISO artefacts. Estimated time to first shippable ISO: approximately 4 weeks. Current status: CI pipeline fully wired; first successful end-to-end build pending.

16.2 Path B — Linux Kernel + Vyro Userland

Path B uses an upstream Linux kernel built with Buildroot and replaces the entire userspace with Vyro-native applications: vyro-compositor (a DRM/KMS Wayland compositor) and five native apps (Calculator, Files, TextEdit, Terminal, Settings) communicating via the Vyro IPC protocol. Estimated time to bootable image: approximately 4 months.

16.3 Path C — From-Scratch Microkernel

Path C is the original research path described throughout this paper. Every line of code is original and every bug found is genuinely novel. The 155 git tags document a real archaeological record of OS development — each tag corresponds to a working state booted in QEMU and verified before tagging. Estimated remaining work to daily-driver status: approximately 60 additional person-months.

17. Production Bug Analysis

One of the most valuable outcomes of building an OS from scratch is encountering bugs that never appear in application programming. The following five bugs were diagnosed through QEMU's GDB stub, CR2 capture instrumentation, and runtime canary probes.

17.1 BSS Corruption — backbuf Pointer [vC.6.11–vC.6.14]

Symptom: comp_present() page-faulted with CR2 = 0xFFFFFFFFFFFFFFFF after certain boot paths. A custom page-fault handler added in vC.6.11.0 captured the faulting address; the value exactly equals the sign-extension of int32 -1, consistent with a 4-byte write of 0xFFFFFFFF to a 64-bit pointer. The guilty write was not identified before the compositor pointer was relocated to fixed physical 0x1000000 in .rodata (vC.6.12.3). Canary instrumentation (vC.6.14) will catch the writer if it recurs.

17.2 sleep_ms Deadlock Before sti [vC.6.15]

Symptom: the boot chime and SMP bringup both hung permanently. Root cause: hlt requires RFLAGS.IF = 1; before sti, IF = 0 so the CPU halts forever. Two earlier fix attempts triple-faulted because the pushfq + pop rax inline asm was missing the memory clobber, causing the compiler to emit an incorrect frame pointer offset. Final fix: pushfq + popq %0 with the memory constraint

correctly reads RFLAGS.IF.

17.3 First Keystroke Eaten [vC.6.16]

Symptom: the first character typed after each boot was silently discarded. Root cause: the i8042 controller latches its power-on self-test result (0xAA) in the output buffer. When IRQ1 is unmasked without draining this byte, the first IRQ fires for the stale byte. Fix: keyboard_init() polls the OBF bit up to 16 times before pic_unmask_irq(1).

17.4 GUI Silent Non-Launch [vC.6.17]

Symptom: the desktop never launched automatically. Root cause: fb_init() was called before the bootloader had finished writing the LFB address to 0x0500 — on some QEMU versions the VBE mode-set completes asynchronously. Fix: fb_probe() writes a known pixel and reads it back; kernel_main() retries fb_init() once if the probe fails.

17.5 Mouse Clicks Non-Functional [vC.6.18]

Symptom: the cursor moved correctly but all clicks produced no response. Root cause: QEMU's usb-tablet does not emulate a PS/2 relative device; it delivers absolute coordinates only via the VMware backdoor at port 0x5658. The buttons global was always zero. Fix: mouse_init() probes the backdoor; if present, mouse_poll() is called each render frame.

18. Evaluation

Feature	Status	Version completed
Boot to 64-bit kernel	Done	v1
Text shell (~40 commands)	Done	v1-v2
Physical memory manager	Done	v1
Heap allocator (kmalloc/kfree)	Done	v1
IDT, PIC, all IRQs	Done	v1
IRQ-aware sleep_ms	Done	vC.6.15
Syscall gate (INT 0x80)	Done	v2
ELF64 user-mode loader	Done	v2
Cooperative + preemptive scheduler	Done	v2-v3
VFS + FAT32 read-only	Done	v2 / vC.6.5
TCP/IP stack	Done	v3
TLS 1.3 key schedule (RFC 8448)	Done	v3
9 crypto primitives	Done	v3-v4
Desktop GUI (1024x768)	Done	v2-v4
Glassmorphic compositor	Done	v4

Feature	Status	Version completed
Mouse clicks (VMware backdoor)	Done	vC.6.18
Boot chime (PC speaker)	Done	vC.6.19
BSS canary instrumentation	Done	vC.6.14
SMP AP bringup (INIT/SIPI)	Deferred	v9
Per-process page tables / KASLR	Deferred	v9
GPU / DRM acceleration	Deferred	future
CI ISO build green	Pending	Path A

Table 9. Feature evaluation matrix at Vyro OS version 8.0.

19. Related Work

Vyro OS shares goals with several educational and research OS projects. xv6 (MIT, 2006-present) is a UNIX-v6 reimplementation for RISC-V and x86-64 used in MIT 6.1810; like Vyro it avoids libc, but its scope is intentionally minimal with no graphics or networking. Redox OS (2015-present) is a microkernel OS written in Rust targeting memory safety; unlike Vyro, it uses Rust's standard library. Serenity OS (2018-present) is the closest analogue: a from-scratch Unix-like OS with a full graphical desktop, networking, and a Chromium-based browser. Serenity demonstrates that a solo from-scratch OS can reach daily-driver quality given sufficient time; Vyro's tri-path strategy is in part inspired by Serenity's eventual decision to layer a Linux-compatible userspace on top of the microkernel. The OSDev Wiki and the cfenollosa tutorial provide the specification Vyro was initially calibrated against.

20. Future Work

The highest-priority near-term items are: (1) SMP AP bringup — send INIT and SIPI IPIs via the LAPIC, establish per-AP stacks, and integrate secondary cores into the scheduler; (2) identify the BSS OOB writer using the canary instrumentation from vC.6.14; (3) first successful Path A CI ISO build; (4) IRQ-aware spinlocks throughout the allocator for true preemptive multitasking; (5) UEFI boot path replacing the legacy BIOS boot sector.

Medium-term goals include full AHCI command queues, NVMe I/O queue pairs, E1000 DMA rings, xHCI HID device enumeration, HD Audio codec initialisation, ACPI AML interpreter, per-process virtual memory with W^X enforcement, a POSIX shell, and a musl libc port. Long-term goals include an Apple Silicon ARM64 port and porting a web browser engine.

21. Conclusion

We have presented Vyro OS, a 64-bit operating system built from scratch across eight major versions and 155 git tags. The system boots visibly in QEMU, renders a full-screen glassmorphic desktop, accepts mouse clicks and keyboard input, plays a boot chime, and implements 25 subsystems — all from first principles without any external library. The development history exposed five non-trivial production bugs, each requiring systematic runtime instrumentation to diagnose.

The tri-path architecture introduced in v7 provides a pragmatic path to a shippable product on a 4-week timeline (Path A) while the research microkernel continues to evolve independently (Path C). Vyro OS at v8.0 is real software — 53,000 lines written from scratch, booted and verified at every step, with every bug fixed in the open and every version tagged and documented.

Acknowledgements

The cfenollosa os-tutorial repository provided the initial structure and the concrete goal of building a shell-capable OS one lesson at a time. The OSDev Wiki (wiki.osdev.org) was an irreplaceable

community resource throughout the project. The QEMU project's source code, particularly hw/input/vmmouse.c, was the definitive reference for the VMware backdoor protocol. The Linux kernel source served as the architecture oracle referenced in the coding rules. Serenity OS demonstrated that a single developer can build a complete graphical operating system from scratch.

References

[1] cfenollosa. *os-tutorial: How to create an OS from scratch.* GitHub, 2016.

[2] Intel Corporation. *Intel 64 and IA-32 Architectures Software Developer's Manual, Vol. 3A.* 2024.

[3] D. Ritchie, K. Thompson. *The UNIX Time-Sharing System.* CACM 17(7):365-375, 1974.

[4] MIT PDOS. *xv6: A Simple, Unix-like Teaching Operating System.* MIT 6.1810, 2023.

[5] E. Rescorla. *The Transport Layer Security (TLS) Protocol Version 1.3.* RFC 8446, IETF, 2018.

[6] Y. Nir, A. Langley. *ChaCha20 and Poly1305 for IETF Protocols.* RFC 8439, IETF, 2018.

[7] D. J. Bernstein. *Curve25519: New Diffie-Hellman Speed Records.* LNCS 3958, Springer, 2006.

[8] H. Krawczyk, P. Eronen. *HMAC-based Extract-and-Expand Key Derivation Function (HKDF).* RFC 5869, IETF, 2010.

[9] VMware, Inc. *VMware Backdoor I/O Port.* Open VM Tools source code, 2008.

[10] QEMU Project. *QEMU: A Fast and Portable Dynamic Translator.* USENIX ATC FREENIX, 2005.

[11] VESA Cooperative. *VESA BIOS Extension Core Functions Standard v3.0.* 1998.

[12] A. Kling. *Serenity OS.* GitHub, 2018. https://github.com/SerenityOS/serenity

[13] A. Tanenbaum, et al. *Can We Make Operating Systems Reliable and Secure?* IEEE Computer, 2006.

[14] P. A. McKenney. *Is Parallel Programming Hard, And, If So, What Can You Do About It?* kernel.org, 2023.

[15] NIST. *FIPS PUB 180-4: Secure Hash Standard.* 2015.

[16] D. R. L. Brown. *SEC 1: Elliptic Curve Cryptography.* Certicom Research / SECG, 2009.

[17] T. Dierks, E. Rescorla. *The TLS Protocol Version 1.2.* RFC 5246, IETF, 2008.

[18] OSDev Wiki. *OSDev.org community reference wiki.* https://wiki.osdev.org

[19] J. Corbet, A. Rubini, G. Kroah-Hartman. *Linux Device Drivers, 3rd edition.* O'Reilly, 2005.

[20] A. S. Tanenbaum. *Modern Operating Systems, 4th edition.* Pearson, 2014.